



**Министерство науки и высшего образования Российской
Федерации**
**Федеральное государственное бюджетное образовательное
учреждение высшего образования**
**«Московский государственный технический университет
имени Н.Э. Баумана**
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа № 2
по дисциплине «Анализ алгоритмов»

Тема Алгоритмы умножения матриц

Студент Пантелеев Василий Максимович

Группа ИУ7-52Б

Преподаватели Строганов Дмитрий Владимирович

Москва, 2024

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ	2
ВВЕДЕНИЕ	3
1 Аналитическая часть	4
1.1 Стандартный алгоритм	4
1.2 Алгоритм Винограда	4
2 Конструкторская часть	6
2.1 Описание алгоритмов	6
2.2 Модель вычислений	12
2.3 Вычисление трудоемкости алгоритмов	12
2.3.1 Стандартный алгоритм	13
2.3.2 Алгоритм Винограда	13
2.3.3 Оптимизированный алгоритм Винограда	13
3 Технологическая часть	15
3.1 Средства реализации	15
3.2 Реализация алгоритмов	16
3.3 Функциональные тесты	19
4 Исследовательская часть	20
4.1 Технические характеристики ЭВМ	20
4.2 Сравнение алгоритмов	20
ЗАКЛЮЧЕНИЕ	22
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	23

ВВЕДЕНИЕ

Цель лабораторной работы — выполнить оценки ресурсной эффективности алгоритмов умножения матриц и их реализации. Для достижения поставленной цели необходимо выполнить следующие задачи:

- описать математическую основу стандартного алгоритма и алгоритма Винограда умножения матриц;
- описать модель вычислений;
- выполнить оценку трудоемкости разрабатываемых алгоритмов;
- реализовать разработанные алгоритмы в программном обеспечении с 2 режимами работы (одиночного расчета и массивованного замера процессорного времени выполнения реализации каждого алгоритма);
- выполнить замеры процессорного времени выполнения реализации разработанных алгоритмов в зависимости от варьируемого линейного размера матриц;
- выполнить сравнительный анализ рассчитанных трудоемкостей и результатов замера процессорного времени выполнения реализации трех алгоритмов с учетом лучшего и худшего случаев по трудоемкости.

1 Аналитическая часть

В данном разделе будут рассмотрены теоретические аспекты алгоритмов умножения матриц.

1.1 Стандартный алгоритм

Стандартный алгоритм умножения матриц A и B предполагает, что для матриц размеров $m \times n$ и $n \times p$ результирующая матрица C будет иметь размер $m \times p$. Элементы результирующей матрицы C вычисляются по следующей формуле:

$$c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj}, \quad (1.1)$$

где c_{ij} — элемент матрицы C , полученный умножением i -й строки матрицы A и j -го столбца матрицы B .

1.2 Алгоритм Винограда

Алгоритм Винограда является оптимизированным методом умножения матриц, который уменьшает количество операций умножения за счёт предварительных вычислений. В этом алгоритме для ускорения используются дополнительные массивы `row_factor` и `col_factor`.

Пусть A — матрица размером $m \times n$, B — матрица размером $n \times p$, и C — результирующая матрица размером $m \times p$.

Рассчитываются вспомогательные массивы:

$$\text{row_factor}_i = \sum_{k=1}^{\lfloor n/2 \rfloor} a_{i,2k-2} \cdot a_{i,2k-1}, \quad (1.2)$$

$$\text{col_factor}_j = \sum_{k=1}^{\lfloor n/2 \rfloor} b_{2k-2,j} \cdot b_{2k-1,j}. \quad (1.3)$$

Элементы результирующей матрицы C вычисляются следующим образом:

$$c_{ij} = -\text{row_factor}_i - \text{col_factor}_j + \sum_{k=1}^{\lfloor n/2 \rfloor} \left((a_{i,2k-2} + b_{2k-1,j}) \cdot (a_{i,2k-1} + b_{2k-2,j}) \right). \quad (1.4)$$

Если n нечётное, необходимо добавить корректирующий член для учёта последнего элемента строки и столбца:

$$c_{ij}+ = a_{i,n-1} \cdot b_{n-1,j}. \quad (1.5)$$

Вывод

В данном разделе были рассмотрены теоретические аспекты алгоритмов умножения матриц.

2 Конструкторская часть

В данном разделе будут приведены описания алгоритмов умножения матриц, а также модель вычисления трудоемкости алгоритмов.

2.1 Описание алгоритмов

На вход алгоритмов подаются 2 матрицы - a и b , на выходе – матрица c , результат умножения матриц a и b . В случае если число столбцов первой матрицы не равно числу строк второй, программа выведет сообщение об ошибке.

На схемах 2.1— 2.3 приведены описания алгоритмов умножения матриц: стандартный алгоритм, алгоритм Винограда, оптимизированный алгоритм Винограда.

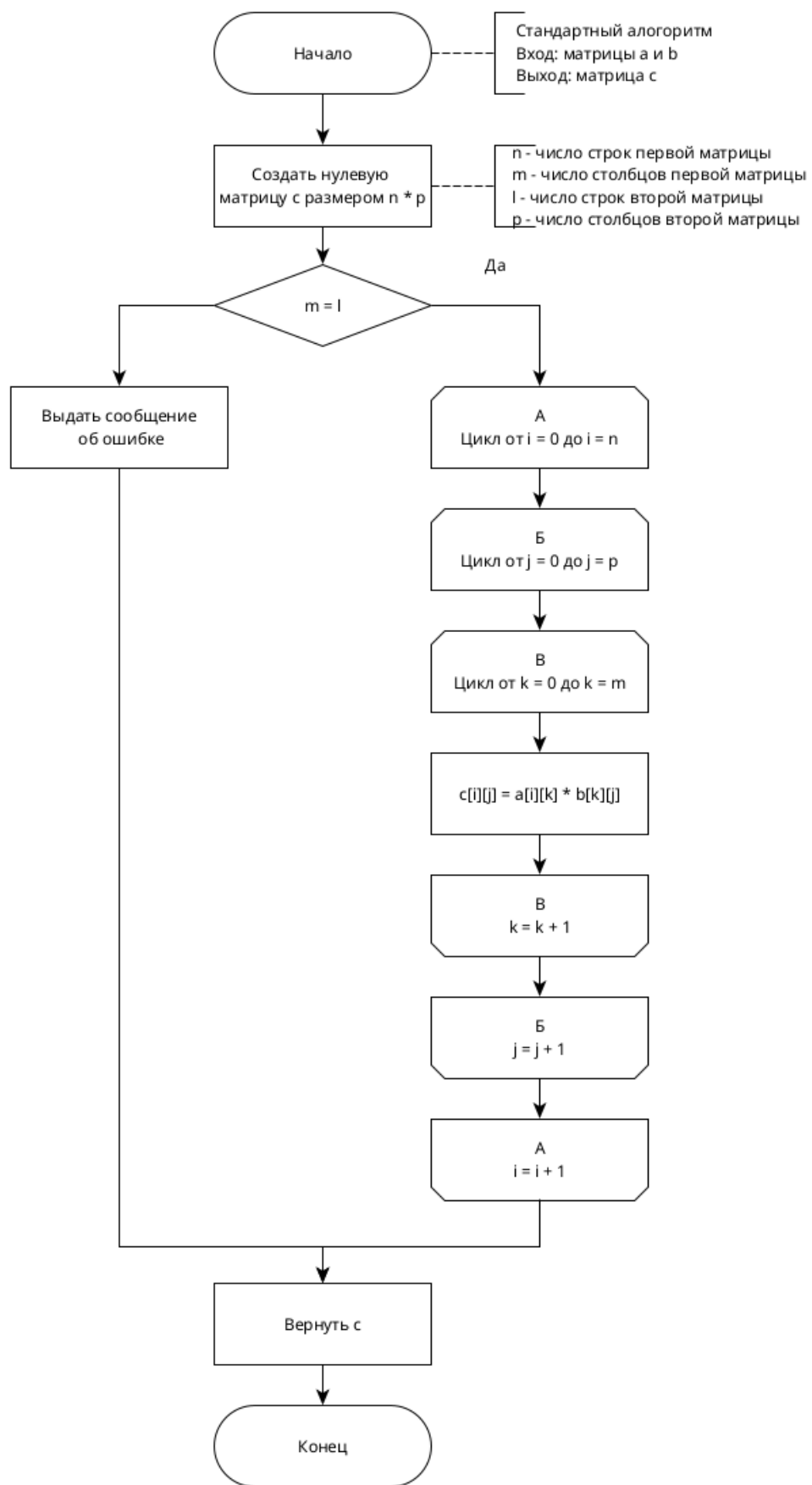
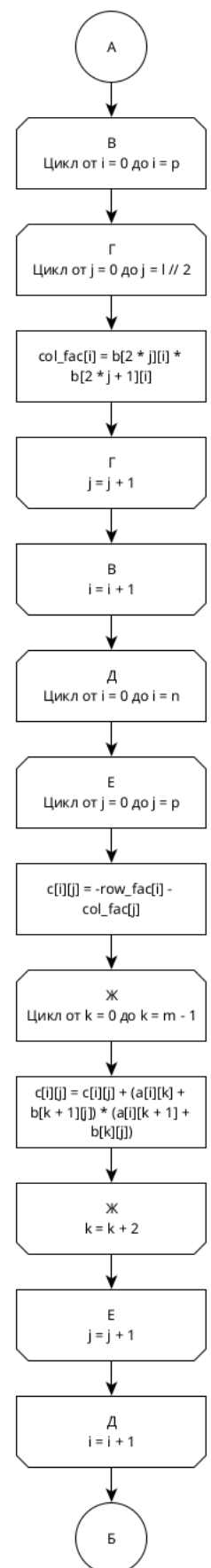
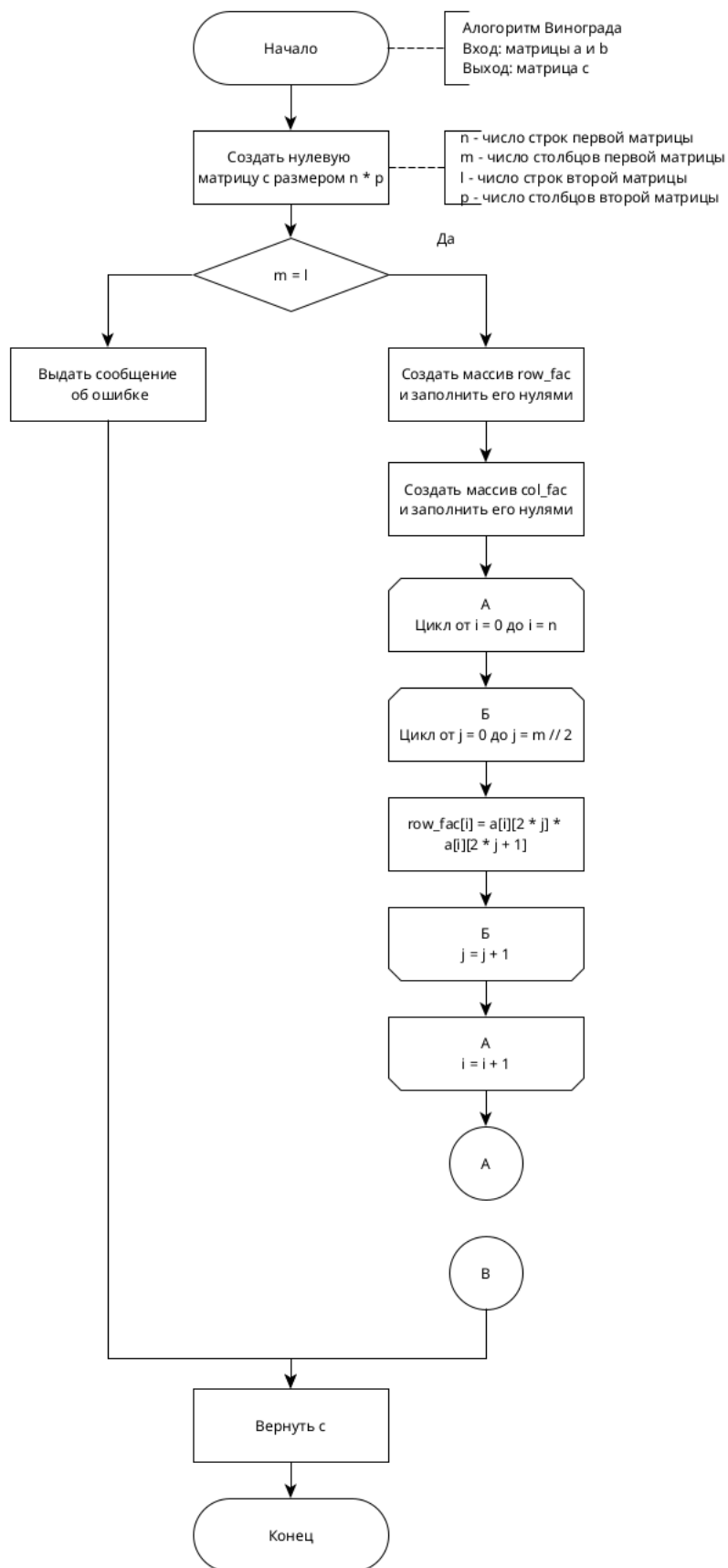


Рисунок 2.1 — Описание стандартного алгоритма умножения матриц



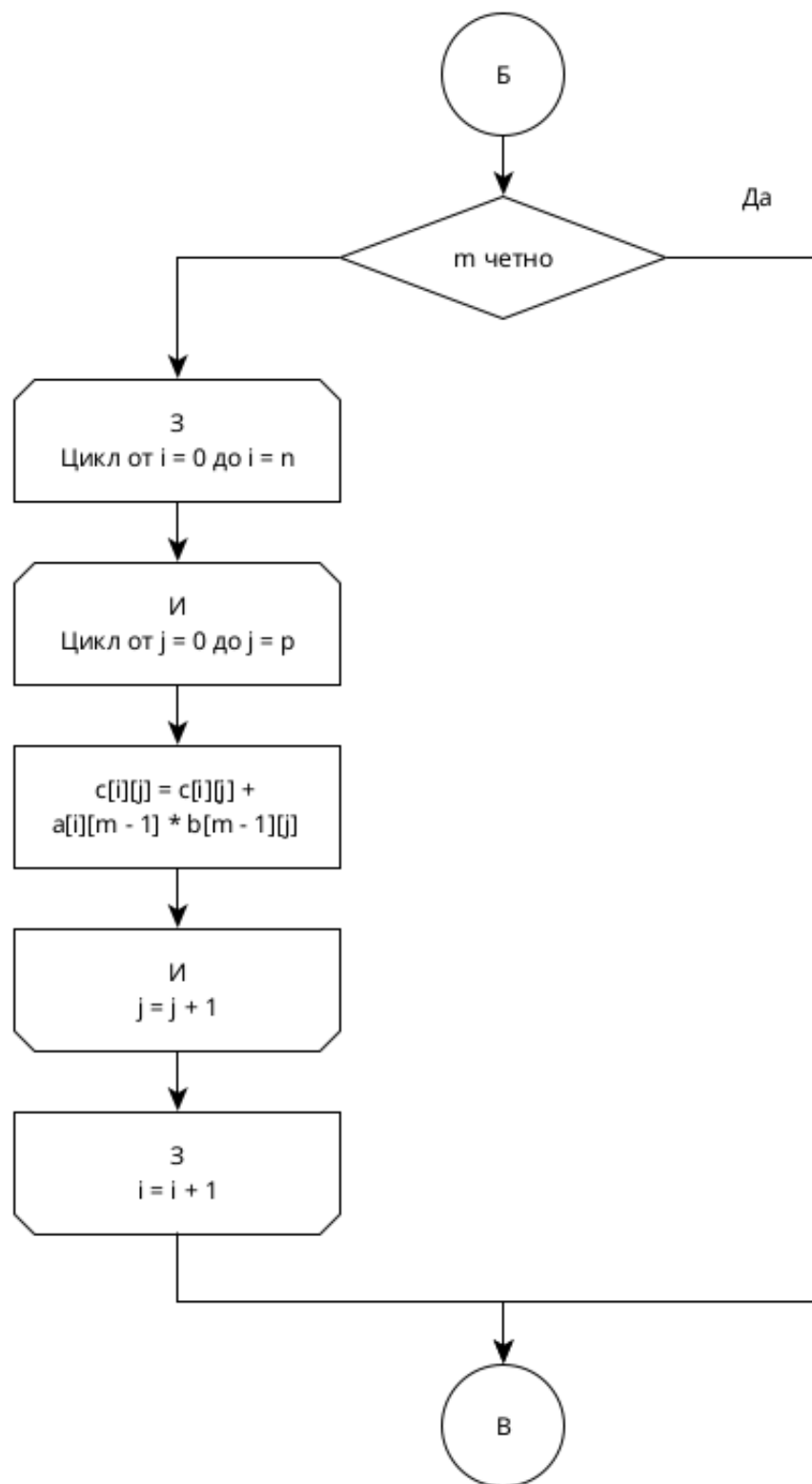
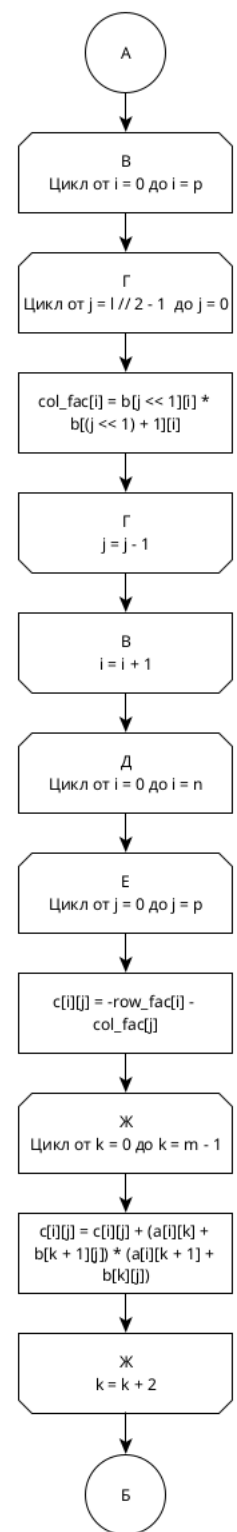
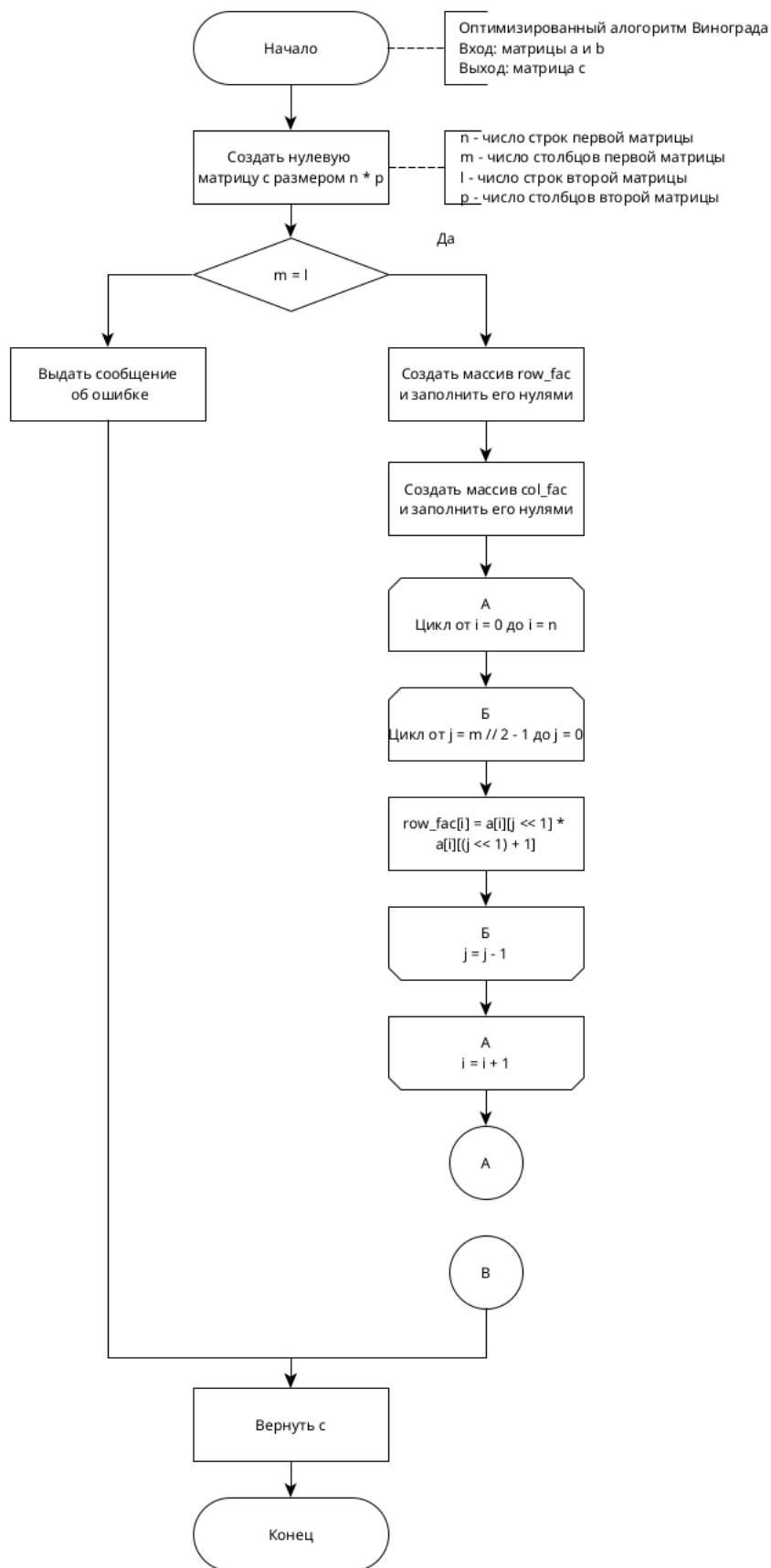


Рисунок 2.2 — Описание алгоритма Винограда умножения матриц



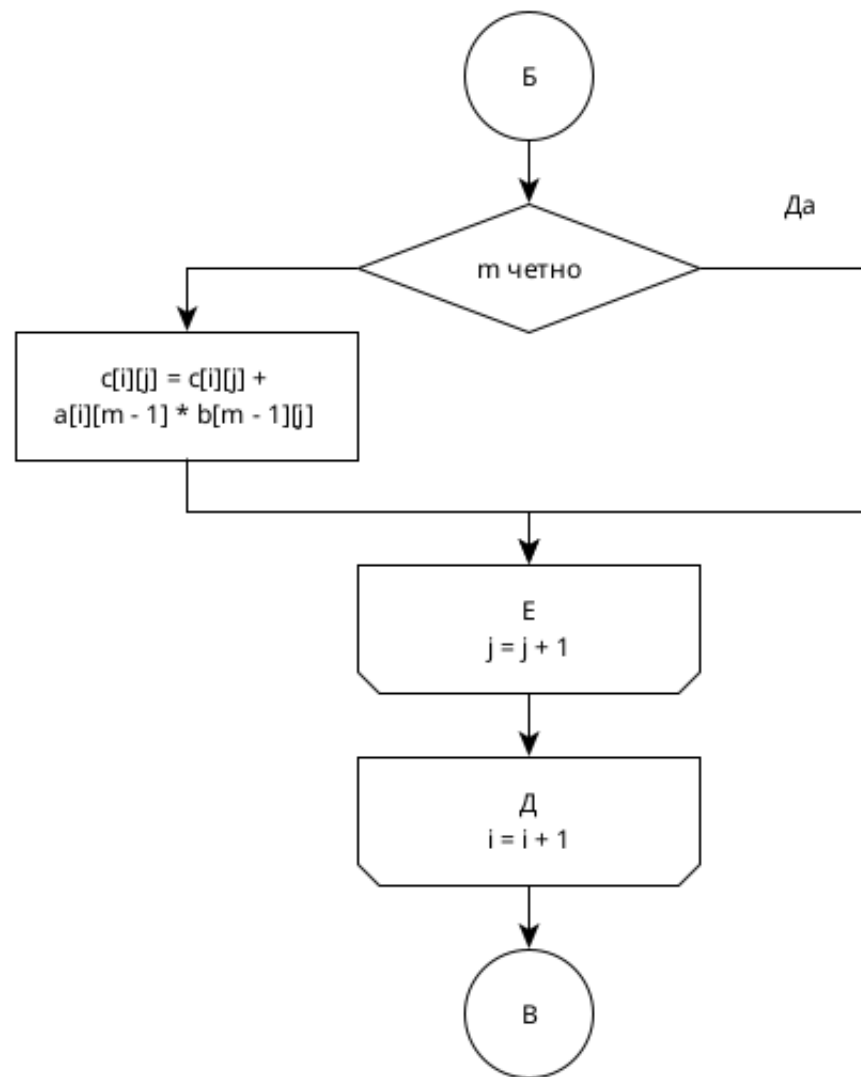


Рисунок 2.3 — Описание оптимизированного алгоритма Винограда умножения матриц

Оптимизация алгоритма Винограда по варианту:

- двоичный сдвиг вместо умножения на 2;
- объединение III и IV частей алгоритма Винограда;
- введение декремента при вычислении вспомогательных массивов.

2.2 Модель вычислений

Для вычисления трудоемкостей алгоритмов необходимо ввести модель вычислений:

- операции $+$, $-$, $[]$, $++$, $--$, $==$, $!=$, $<$, $>$, $<=$, $>=$ имеют трудоемкость 1;
- операции $*$, $/$, $\%$, $/=$, $*=$, $\%=$ имеют трудоемкость 2;
- трудоемкость условного оператора `if <condition> then A else B` рассчитывается, как (2.1);

$$f_{if} = f_{condition} + \begin{cases} f_A, & \text{если условие выполняется,} \\ f_B, & \text{иначе.} \end{cases} \quad (2.1)$$

- трудоемкость цикла `for (init; condition; increment) { body }` рассчитывается, как (2.2);

$$f_{for} = f_{init} + f_{condition} + N \cdot (f_{body} + f_{condition} + f_{increment}) \quad (2.2)$$

где N – количество итераций цикла;

- трудоемкость вызова функции равна 0.

2.3 Вычисление трудоемкости алгоритмов

Далее представлен расчет трудоемкостей алгоритмов умножения матриц на основе модели. Для расчета представлены такие алгоритмы, как стандартный алгоритм умножения матриц, алгоритм Винограда и оптимизированный алгоритм Винограда.

Проверка входных данных и инициализация результирующей матрицы в расчетах учитываться не будут, так как данные действия не являются самыми трудоемкими и являются общими для всех алгоритмов.

Обозначения размерностей:

- n – число строк первой матрицы;
- m – число столбцов первой матрицы / строк второй матрицы;
- l – число столбцов второй матрицы.

2.3.1 Стандартный алгоритм

Расчет трудоемкости стандартного алгоритма:

- трудоемкость внешнего цикла по i , $f_i = 2 + n(2 + f_{body_i})$;
- трудоемкость цикла по j , $f_j = 2 + l(2 + f_{body_j})$;
- трудоемкость цикла по k , $f_k = 2 + m(2 + f_{body_k})$;
- трудоемкость тела цикла по k , $f_{body_k} = 10$,

где $f_{body_i} = f_j$, $f_{body_j} = f_k$

$$f_{std} = 2 + n(4 + l(4 + 12m)) = 12nml + 4mn + 4n + 2 \approx 12nml$$

2.3.2 Алгоритм Винограда

Расчет трудоемкости алгоритма Винограда:

- создание и инициализация вспомогательных массивов, $n + l$
- заполнение первого массива, $f_1 = 2 + n(2 + 4 + \frac{m}{2}(4 + 14))$
- заполнение второго массива, $f_2 = 2 + l(2 + 4 + \frac{m}{2}(4 + 14))$
- заполнение четных размерностей, $f_3 = 2 + n(4 + l(2 + 7 + 4 + \frac{m}{2}(4 + 26)))$
- трудоемкость дополнения если m нечетное $f_4 = f_{cond}$.

$$f_{cond} = \begin{cases} 3, & \text{если } m \text{ четно (л.с.)}, \\ 3 + 2 + n(4 + l(2 + 12)), & \text{иначе (х.с.)}. \end{cases} \quad (2.3)$$

$$f_{wyn} = \begin{cases} 9 + 11n + 7l + 13ln + 9nm + 9lm + 15nml, & \text{(л.с.)}, \\ 11 + 15n + 7l + 27ln + 9nm + 9lm + 15nml, & \text{(х.с.)}, \end{cases} \quad (2.4)$$

2.3.3 Оптимизированный алгоритм Винограда

Расчет трудоемкости оптимизированного алгоритма Винограда:

- создание и инициализация вспомогательных массивов, $n + l$
- заполнение первого массива, $f_1 = 2 + n(2 + 5 + \frac{m}{2}(2 + 12))$
- заполнение второго массива, $f_2 = 2 + l(2 + 5 + \frac{m}{2}(2 + 12))$
- заполнение матрицы c , $f_3 = 2 + n(4 + l(2 + 7 + 4 + \frac{m}{2}(4 + 22))) + f_4$
- трудоемкость дополнения если m нечетное $f_4 = f_{cond}$.

$$f_{cond} = \begin{cases} 3, & \text{если } m \text{ четно (л.с.)}, \\ 3 + 12, & \text{иначе (х.с.)}. \end{cases} \quad (2.5)$$

$$f_{optwyn} = \begin{cases} 6 + 12n + 8l + 16ln + 7nm + 7lm + 13nml, & \text{(л.с.)}, \\ 6 + 12n + 8l + 28ln + 7nm + 7lm + 13nlm, & \text{(х.с.)}, \end{cases} \quad (2.6)$$

Вывод

В данном разделе были представлены схемы алгоритмов умножения матриц, а также оценены трудоемкости для данных алгоритмов в лучшем и худшем случаях. Алгоритм Винограда имеет большую трудоемкость по сравнению со стандартным алгоритмом. Оптимизированный алгоритм имеет меньшую трудоемкость по сравнению с неоптимизированным.

3 Технологическая часть

В этом разделе обоснованы средства реализации, а так же представлены реализации алгоритмов и функциональные тесты.

3.1 Средства реализации

Для реализации алгоритмов в этой лабораторной работы был выбран язык *Python*^[1] по следующим причинам:

- python предоставляет необходимые структуры данных, такие как списки (массивы) и вложенные списки (матрицы);
- поддерживаемые библиотеки, такие как *matplotlib*^[2] позволяют визуализировать результаты исследования.

3.2 Реализация алгоритмов

В листингах 3.1 — 3.3 представлены реализации алгоритмов.

Листинг 3.1 — Стандартный алгоритм умножения матриц

```
def standard_multiply(a, b):
    rows_a = len(a)
    rows_b = len(b)

    if rows_a == 0 or rows_b == 0:
        return None

    cols_a = len(a[0])
    cols_b = len(b[0])

    if cols_a != rows_b or cols_a == 0 or cols_b == 0:
        return None

    c = [[0] * cols_b for _ in range(rows_a)]

    for i in range(rows_a):
        for j in range(cols_b):
            for k in range(cols_a):
                c[i][j] += a[i][k] * b[k][j]

    return c
```

Листинг 3.2 — Алгоритм Винограда

```
def winograd_multiply(a, b):
    rows_a = len(a)
    rows_b = len(b)

    if rows_a == 0 or rows_b == 0:
        return None

    cols_a = len(a[0])
    cols_b = len(b[0])

    if cols_a != rows_b or cols_a == 0 or cols_b == 0:
        return None

    c = [[0] * cols_b for _ in range(rows_a)]

    row_factor = [0] * rows_a
    col_factor = [0] * cols_b

    for i in range(rows_a):
        for j in range(cols_a // 2):
            row_factor[i] += a[i][2 * j] * a[i][2 * j + 1]

    for i in range(cols_b):
        for j in range(rows_b // 2):
            col_factor[i] += b[2 * j][i] * b[2 * j + 1][i]

    for i in range(rows_a):
        for j in range(cols_b):
            c[i][j] += -row_factor[i] - col_factor[j]

            for k in range(cols_a // 2):
                c[i][j] += (a[i][2 * k] + b[2 * k + 1][j]) * (a[i][2 * k + 1] + b[2 * k][j])

    if cols_a % 2 != 0:
        for i in range(rows_a):
            for j in range(cols_b):
                c[i][j] += a[i][cols_a - 1] * b[rows_b - 1][j]

    return c
```

Листинг 3.3 — Оптимизированный алгоритм Винограда

```
def optimized_winograd_multiply(a, b):
    rows_a = len(a)
    rows_b = len(b)

    if rows_a == 0 or rows_b == 0:
        return None

    cols_a = len(a[0])
    cols_b = len(b[0])

    if cols_a != rows_b or cols_a == 0 or cols_b == 0:
        return None

    c = [[0] * cols_b for _ in range(rows_a)]

    row_factor = [0] * rows_a
    col_factor = [0] * cols_b

    for i in range(rows_a):
        j = cols_a // 2 - 1
        while j >= 0:
            row_factor[i] += a[i][j << 1] * a[i][(j << 1) + 1]
            j -= 1

    for i in range(cols_b):
        j = rows_b // 2 - 1
        while j >= 0:
            col_factor[i] += b[j << 1][i] * b[(j << 1) + 1][i]
            j -= 1

    for i in range(rows_a):
        for j in range(cols_b):
            c[i][j] += -row_factor[i] - col_factor[j]

            for k in range(cols_a // 2):
                c[i][j] += (a[i][k << 1] + b[(k << 1) + 1][j]) * (a[i][(k << 1) + 1] + b[k << 1][j])

            if cols_a % 2 != 0:
                c[i][j] += a[i][cols_a - 1] * b[rows_b - 1][j]

    return c
```

3.3 Функциональные тесты

В таблице 3.1 приведены функциональные тесты, на которых тестировалась программа.

Таблица 3.1 — Функциональные тесты, на которых были протестированы алгоритмы (в результате – матрица с).

Матрица a	Матрица b	Результат
$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$	$\begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$	$\begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}$
$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$	$\begin{pmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{pmatrix}$	$\begin{pmatrix} 58 & 64 \\ 139 & 154 \end{pmatrix}$
$(1 \ 2 \ 3)$	$\begin{pmatrix} 4 \\ 5 \\ 6 \end{pmatrix}$	(32)
$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$	$\begin{pmatrix} 9 & 8 & 7 \\ 6 & 5 & 4 \\ 3 & 2 & 1 \end{pmatrix}$	$\begin{pmatrix} 30 & 24 & 18 \\ 89 & 69 & 54 \\ 138 & 114 & 90 \end{pmatrix}$
$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} 5 & 6 & 7 \\ 8 & 9 & 10 \\ 11 & 12 & 13 \end{pmatrix}$	$\begin{pmatrix} 5 & 6 & 7 \\ 8 & 9 & 10 \\ 11 & 12 & 13 \end{pmatrix}$
$()$	$()$	Ошибка
$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$	$(5 \ 6 \ 7)$	Ошибка

Вывод

В технологической части были представлены листинги реализованных алгоритмов, включая стандартный алгоритм умножения матриц, а также алгоритм Винограда и оптимизированный алгоритм Винограда. Также были проведены функциональные тесты, которые подтвердили корректность работы алгоритмов на различных входных данных, включая граничные случаи. Результаты тестов показали, что разработанные реализации соответствуют поставленным требованиям.

4 Исследовательская часть

В этом разделе приведен сравнительный анализ алгоритмов по времени выполнения.

4.1 Технические характеристики ЭВМ

Все замеры проводились на ЭВМ при использовании микроконтроллера *STM32*[3]. Характеристики ЭВМ приведены ниже:

- процессор: AMD Ryzen™ 7–5800H with Radeon™ Graphics × 16;
- ОС: 64-разрядная операционная система Ubuntu 24.04.1 LTS;
- оперативная память: 16,0 ГБ.

4.2 Сравнение алгоритмов

В таблице 4.1 приведено время работы алгоритмов в секундах. Для повышения точности результатов проводилось 200 замеров.

Таблица 4.1 — Сравнение алгоритмов умножения матриц по времени выполнения

Размер квадратной матрицы	Стандартный	Виноград	Оптимизированный
10	0.00015	0.00017	0.00020
30	0.0020	0.0016	0.0020
50	0.009	0.009	0.009
70	0.017	0.016	0.016
90	0.035	0.032	0.030
110	0.041	0.037	0.036

На графике 4.1 приведена визуализация результатов.

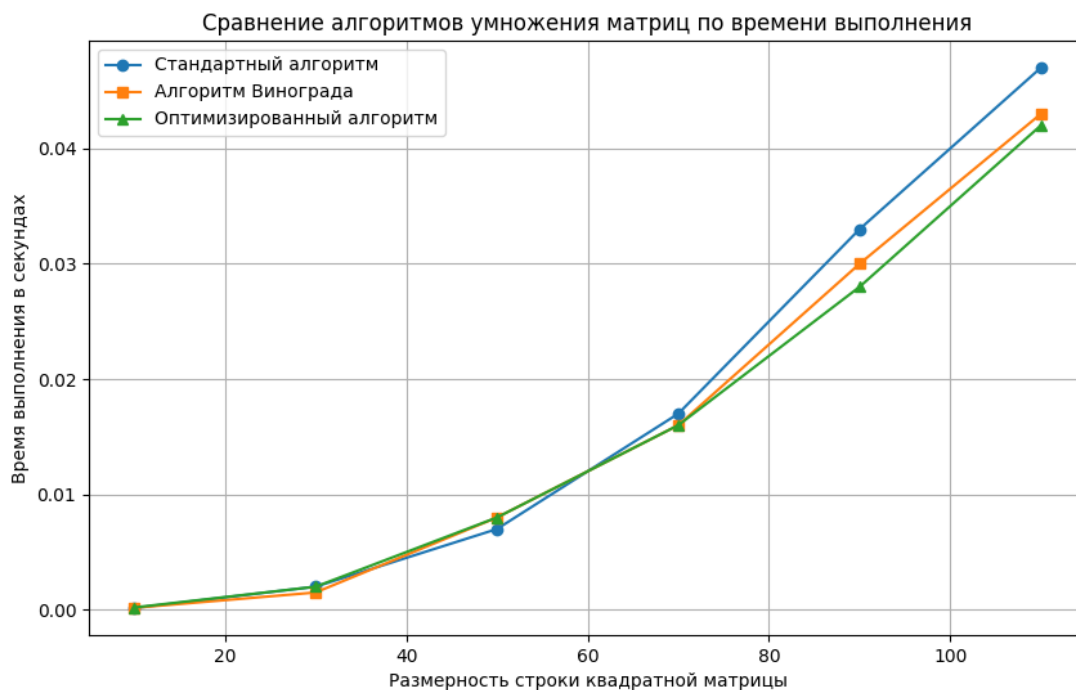


Рисунок 4.1 — Сравнения для каждого элемента в алгоритме полного перебора

Вывод

Хотя оптимизированный алгоритм Винограда имеет большую теоретическую сложность по сравнению со стандартным, он работает быстрее при умножении матриц размерностью от 40 элементов (примерно на 10%). Оптимизированный алгоритм работает быстрее неоптимизированной версии на размерностях матриц больше 70.

ЗАКЛЮЧЕНИЕ

В результате работы были подтверждены различия в эффективности алгоритмов. Теоретическая оценка показала различия в сложности представленных алгоритмов. Алгоритм Винограда имеет большую временную сложность по сравнению со стандартными алгоритмами. Несмотря на это исследование показало, что алгоритм Винограда и его оптимизированная версия в большинстве случаев выполняются быстрее стандартного алгоритма (примерно на 10%). Оптимизация, предложенная Виноградом, уменьшает количество операций умножения и сложения за счёт предварительных вычислений, что сокращает общее время выполнения. Оптимизация алгоритма Винограда позволила ещё больше сократить время выполнения, подтвердив эффективность предложенных улучшений.

В ходе выполненной работы были выполнены следующие задачи:

- описана математическая основа стандартного алгоритма умножения матриц и алгоритма Винограда;
- описана модель вычислений трудоемкости алгоритмов;
- реализованы различные алгоритмы умножения матриц;
- выполнена оценка трудоемкости алгоритмов, согласно которой самый простой алгоритм – стандартный;
- проведено сравнение алгоритмов по времени выполнения;
- описаны и обоснованы полученные результаты.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Welcome to Python – [Электронный ресурс]. URL: <https://https://www.python.org> (дата обращения: 23.10.2024).
2. Библиотека matplotlib – [Электронный ресурс]. URL: <https://matplotlib.org/> (дата обращения: 23.10.2024).
3. STM32 PDF Documentation [Электронный ресурс]. URL: <https://www.st.com/en/microcontrollers-microprocessors/stm32-arm-cortex-mpus/documentation.html> (дата обращения: 23.10.2024).